

SMART ANTI-BUNKING ATTENDANCE SYSTEM USING FACE RECOGNITION

PROJECT WORK DONE BY

NAME: AISHWARYA. R

REG. NO.: 2413322078002

Under the guidance of

Dr. D. Sridevi M.C.A., M.Phil., Ph.D.



DEPARTMENT OF COMPUTER SCIENCE

ETHIRAJ COLLEGE FOR WOMEN

March – 2026

SMART ANTI-BUNKING ATTENDANCE SYSTEM USING FACE RECOGNITION

PROJECT WORK DONE BY

NAME: AISHWARYA. R

REG. NO.: 2413322078002

Under the guidance of

Dr. D. Sridevi M.C.A., M.Phil., Ph.D.



**DEPARTMENT OF COMPUTER SCIENCE
ETHIRAJ COLLEGE FOR WOMEN**

March – 2026

CERTIFICATE

This is to certify that project work entitled

SMART ANTI-BUNKING ATTENDANCE SYSTEM USING FACE RECOGNITION

is a Bonafide record of work done by

AISHWARYA. R

2413322078002

in partial fulfillment of the award of the degree of

MASTER OF COMPUTER SCIENCE

University of Madras

Head of the Department

Ms. K. Jane Nithya M.Sc., M.Phil.,

Department of Computer Science

Ethiraj College For Women

Faculty Guide

Dr. D. Sridevi M.C.A., M.Phil., Ph.D.

Department of Computer Science

Ethiraj College For Women

Submitted for the viva – voce examination held on _____

EXTERNAL EXAMINER

INTERNAL EXAMINER

DECLARATION

I hereby declare that this project work entitled “**SMART ANTI-BUNKING ATTENDANCE SYSTEM USING FACE RECOGNITION**” submitted to Ethiraj College for Women – Chennai, an Autonomous Institution, Affiliated to University of Madras, Chennai, is a record of original work done by me under the guidance of **Dr. D. Sridevi** and that this project has not formed the basis for the award of any degree / diploma / associate ship / fellowship or similar to any candidate in any university.

Place:

Date:

Signature of the Student

Countersigned by

Dr. D. Sridevi

ACKNOWLEDGEMENT

First and foremost, I thank the Almighty God, who has showered his blessings on me throughout the period of development. I dedicate my sincere thanks to my parents for encouraging me and giving me invaluable and moral support to do this project work. I express my sincere thanks wholeheartedly to Dr. S. Uma Gowrie, M.Sc., M.Phil., Ph.D., Principal & Secretary, who had given me the prestigious opportunity of being the student of this institution. I am very much indebted to them, who had been the circumstances require success.

My sincere thanks to Ms. K. Jane Nithya, Head of the Department, who had provided me all the facilities to complete my project. “Guidance is the capacity to translate vision into reality”. I dedicate my heartfelt thanks to my guide, Dr. D. Sridevi, Associate Professor, Department of Computer Science, who has been the source of power and price for success and who guided us with initiation and effort towards project development.

Lastly, my sincere thanks to everyone who helped me in the completion of this project.

AISHWARYA R

TABLE OF CONTENTS

			Page No.
		Abstract	1
Chapter	I	Introduction	2
	1.1	Objective Of The Project	2
	1.2	Scope Of The Project	2
Chapter	II	System Analysis	4
	2.1	Existing System	5
	2.1.1	Disadvantages Of The Existing System	5
	2.2	Proposed System	6
	2.2.1	Advantages Of The Proposed System	6
Chapter	III	Project Description	8
	3.1	Project Modules	8
	3.1.1	Admin Dashboard And Its Components	8
	3.1.2	Faculty Dashboard And Its Components	10
	3.1.3	Student Dashboard And Its Components	8
Chapter	IV	System Requirements	11
	4.1	Hardware Specifications	11
	4.2	Software Specifications	11
Chapter	V	System Design	12
	5.1	System Architecture Diagram	12
	5.2	Use Case Diagram	13
Chapter	VI	Functional Requirements	14
	6.1	Table Structure	14
Chapter	VII	Software Environment	15
	7.1	Backend Programming Language: Python	15
	7.2	Frontend Framework: Streamlit	15

	7.3	Core Libraries And Modules	16
	7.4	Database Management System: MYSQL	17
Chapter	VIII	Software Implementation	18
Chapter	IX	Sample Screens	38
Chapter	X	User Manual	44
Chapter	XI	Conclusion	45
Bibliography			47

ABSTRACT

Traditional attendance management in educational ecosystems relies heavily on manual, time-consuming processes such as verbal roll calls and physical sign-in sheets. These archaic methods are fundamentally flawed, making institutions highly vulnerable to proxy attendance, data entry errors, and a significant loss of valuable instructional time. More importantly, existing administrative frameworks consistently fail to address the pervasive issue of "class bunking." Guardians are typically notified of attendance shortages only at the end of an academic semester, by which time corrective disciplinary action is often too late.

To resolve these systemic inefficiencies, this project introduces a Smart Anti-Bunking Attendance System powered by high-accuracy facial recognition technology and real-time automated communication. Utilizing a live webcam feed, the system actively scans the classroom environment, isolates facial features, and maps biometric encodings against a secure, centralized MySQL database to log student attendance instantaneously. This completely eradicates the possibility of fraudulent proxies, as physical presence is computationally verified by the system's deep learning architecture.

Furthermore, to bridge the critical communication gap between the educational institution and parents, the system integrates a dynamic WhatsApp notification module. The moment a lecture concludes, the system autonomously calculates the absentee roster and dispatches immediate, direct alerts to the registered mobile numbers of the students' guardians. By synergizing advanced computer vision with instant external communication capabilities, this proposed system not only ensures strict accountability and eliminates attendance manipulation but also fosters a highly disciplined academic environment by keeping parents actively engaged in their ward's daily educational journey.

CHAPTER I

1 INTRODUCTION

The rapid advancement of computer vision and biometric technologies has fundamentally transformed administrative frameworks across various computing domains. However, educational institutions frequently still rely on antiquated, manual paradigms for attendance tracking, such as verbal roll calls and paper-based sign-in ledgers. These traditional methodologies are not only highly susceptible to human error and deliberate manipulation—such as proxy attendance—but they also passively consume a substantial portion of allotted lecture time. More critically, these disjointed systems fail to provide real-time behavioural insights, allowing the systemic issue of "class bunking" to persist unnoticed until the culmination of the academic term.

This project bridges the gap between modern biometric security and proactive academic administration by developing a Smart Anti-Bunking Attendance System. Utilizing deep learning-based facial recognition, the system autonomously authenticates student presence in real-time. By coupling this biometric verification with an automated communication pipeline, the architecture immediately alerts parents regarding unauthorized absences, fundamentally shifting attendance management from a passive, delayed recording mechanism to an active, transparent, and immediate disciplinary tool.

1.1 OBJECTIVE OF THE PROJECT

The primary objective of this project is to architect and deploy a highly accurate, automated attendance tracking pipeline that entirely eliminates manual data entry and recovers valuable instructional time. A critical secondary objective is to implement a robust, foolproof anti-bunking mechanism. By integrating a dynamic messaging API, the system aims to dispatch instant WhatsApp notifications directly to guardians the moment a student is identified as absent after a lecture scan. This ensures immediate parental awareness through a platform with high open rates, definitively eradicates the possibility of proxy attendance through strict biometric enforcement, and fosters a rigorous culture of accountability within the institutional environment.

1.2 SCOPE OF THE PROJECT

The scope of this project encompasses the end-to-end development of a centralized biometric attendance software system tailored for college and university ecosystems. It includes the localized capture and processing of real-time webcam video feeds to extract, process, and match high-dimensional facial encodings against a normalized relational database. The system provisions secure, role-based

access control interfaces, featuring distinct and isolated dashboards for Administrators, Faculty, and Students to manage master enrolments, monitor live class attendance, and review individual academic standing. Furthermore, the operational scope strictly covers the automated programmatic triggering of external communication protocols to dynamically route absentee alerts to registered parent contact numbers, ensuring a closed-loop network between the classroom and the home.

CHAPTER II

2 SYSTEM ANALYSIS

The Smart Anti-Bunking Attendance System Using Face Recognition operates as a cohesive, end-to-end biometric ecosystem where every interaction is strictly governed by Role-Based Access Control (RBAC) and anchored by a centralized MySQL database. The operational workflow is sequential, ensuring data integrity from the initial point of user enrolment to the final generation of academic reports. Access to the system is authenticated through a secure login portal, which dynamically routes users to isolated dashboards—Administrator, Faculty, or Student, based on their predefined systemic roles.

The administrative lifecycle commences with the "Register Student" module. Authorized personnel input comprehensive student demographic data, academic department affiliations, and the critical parent or guardian WhatsApp contact number. Concurrently, a high-resolution facial photograph of the student is uploaded into the system. The backend computer vision pipeline processes this image to extract a unique 128-dimensional facial encoding. Both the personal metadata and the biometric array are securely committed to the master MySQL database, establishing the immutable foundation required for all subsequent systemic actions.

Day-to-day classroom operations are managed through the Faculty dashboard. At the commencement of a lecture, the faculty member initiates the "Mark Attendance" sequence, which activates the real-time webcam feed. As students enter the optical frame, the system dynamically captures video frames, detects human faces, and computes biometric encodings in real time. These encodings are continuously queried against the master database. Upon a successful algorithmic match, the system executes a secure SQL transaction, instantaneously updating the daily ledger to reflect a "Present" status for the recognized individual.

Upon termination of the video feed, the system transitions into its automated post-session analytical phase. The backend logic queries the MySQL database to cross-reference the comprehensive master enrolment roster against the newly populated daily attendance ledger, systematically isolating students marked as absent. Fulfilling the core anti-bunking mandate, the system leverages the pywhatkit library to execute browser automation. This module dynamically generates and dispatches instant absentee alerts directly to the registered WhatsApp numbers of the guardians, ensuring immediate communication without manual administrative intervention.

Following the session, faculty and administrative users can access the "View Attendance" module. The system retrieves the updated relational data from MySQL and renders it into a comprehensive, interactive graphical data grid. To accommodate offline institutional reporting and archival requirements, the system features a "Download Excel" utility, allowing users to dynamically export the queried database records into a structured spreadsheet format for further analysis.

The final component of the operational lifecycle is the Student Dashboard. Bypassing all administrative and scanning utilities, this portal provides a secure, read-only interface for the end-user. Upon student login, the backend routing logic queries the central database using the unique session credentials to calculate real-time academic standing. The dashboard transparently displays cumulative metrics, including total working days, days present, days absent, and the overall attendance percentage. Throughout this entire sequential pipeline, the MySQL database remains the central nexus, dictating, recording, and authenticating every interaction to maintain absolute operational integrity.

2.1 EXISTING SYSTEM

The existing attendance management system in the vast majority of educational institutions relies heavily on manual, labour-intensive, and antiquated processes. Faculty members typically initiate verbal roll calls at the beginning of a session or circulate physical paper-based sign-in ledgers during the lecture period. In some marginally modernized but still flawed setups, institutions employ RFID identification card swiping or barcode scanners. These traditional systems function merely as passive data collection mechanisms. They record attendance linearly into a localized ledger without any real-time identity validation or active monitoring protocols to ensure the continuous physical presence of the actual registered student.

2.1.1 DISADVANTAGES OF THE EXISTING SYSTEM

The current administrative framework presents several critical vulnerabilities that degrade the academic environment:

- **Unrestricted Class Bunking:** The most significant systemic flaw is the complete inability to actively deter or immediately detect "class bunking." Because attendance shortages are calculated manually and reported in batches only at the end of the month or semester, students can frequently skip classes without any immediate repercussions or parental knowledge.
- **Vulnerability to Proxy Attendance:** Manual sign-in sheets and verbal roll calls are easily and routinely exploited. Students frequently mark attendance or answer roll calls for their absent peers, entirely undermining the integrity of the institution's academic records.

- **Severe Loss of Instructional Time:** Conducting a manual roll call or verifying identities in a classroom of fifty or more students consumes approximately 10 to 15 minutes of valuable lecture time daily, directly impacting the syllabus completion rate.
- **Delayed Guardian Communication:** Parents and guardians remain completely unaware of their ward's absenteeism until formal academic warning letters are dispatched weeks or months later, severely limiting the possibility of proactive disciplinary intervention.

2.2 PROPOSED SYSTEM

The proposed Smart Anti-Bunking Attendance System Using Face Recognition engineers an automated, biometrically secure computer vision pipeline to completely replace manual data entry. Utilizing a live classroom webcam feed, the system continuously captures video frames, detects human faces, and extracts 128-dimensional biometric encodings using deep learning algorithms. These encodings are cross-referenced in real-time against a centralized, secure MySQL database to mark students as present. Upon the completion of the scanning phase, the backend routing logic autonomously cross-references the daily roster to identify missing students. To handle the communication pipeline, the system integrates pywhatkit, a specialized Python automation module. Rather than relying on costly enterprise APIs, this library programmatically interfaces with WhatsApp Web to dynamically generate and dispatch instant absentee alerts directly to the registered guardian's mobile number.

2.2.1 ADVANTAGES OF THE PROPOSED SYSTEM

- **High-Visibility WhatsApp Notifications:** By utilizing WhatsApp—a modern messaging platform with significantly higher open, read, and engagement rates compared to traditional SMS or institutional emails—the system ensures that absentee alerts are practically impossible for parents to ignore, filter as spam, or overlook.
- **Eradication of Proxies:** Biometric facial recognition requires the strict, undeniable physical presence of the individual student, completely neutralizing the threat of proxy attendance and ID card swapping.
- **Time Optimization:** The computer vision system operates efficiently, processing faces in milliseconds, thereby allowing faculty members to dedicate the entirety of the lecture period strictly to academic instruction.

- **Immediate Disciplinary Action:** Real-time data processing and automated communication bridge the gap between the classroom and the home, instantly curbing the culture of class bunking by holding students immediately accountable.

CHAPTER III

3 PROJECT DESCRIPTION

The Smart Anti-Bunking Attendance System Using Face Recognition is structured around a highly modular, role-based architecture. To maintain stringent data security while ensuring operational flexibility, the system divides its functionalities into distinct user modules. This modular approach ensures that users only interact with the specific components required for their institutional roles. The system enforces Role-Based Access Control (RBAC) via a centralized login portal, authenticating credentials against the MySQL database before granting access to the respective dashboards.

3.1 PROJECT MODULES

The application is segmented into three primary modules: Admin, Faculty, and Student. A critical design architectural decision in this system is the unification of the Admin and Faculty dashboards. While their core administrative responsibilities differ, providing both roles with an identical graphical user interface (GUI) and toolset ensures that Administrators can execute necessary actions, such as marking attendance or registering new users, during critical situations or faculty absences without encountering systemic bottlenecks.

3.1.1 ADMIN DASHBOARD AND ITS COMPONENTS

Role of the Admin: The Administrator acts as the overarching system superuser. Their primary responsibility is to oversee the entire biometric infrastructure, ensure data integrity within the master MySQL database, and act as a failsafe for academic operations. The Admin is tasked with handling the initial deployment of the system, managing the master student registry, and intervening in daily operations when a faculty member is incapacitated or facing technical difficulties.

Components: Because the Admin requires full operational parity with the Faculty to handle critical overrides, their dashboard contains the complete suite of system utilities:

- **Register Student Module:** The interface used to input core demographic data, assign academic departments, input guardian WhatsApp contact numbers, and capture the initial biometric photograph to generate the 128-dimensional facial encoding.
- **Mark Attendance Module:** A critical operational component that activates the live webcam feed, runs the real-time computer vision algorithms, and commits "Present" statuses directly to the database. The Admin can utilize this to run a session if the designated faculty is unavailable.

- **Automated Communication Trigger:** Integrated within the attendance module, this component utilizes pywhatkit browser automation to dispatch absentee alerts to parents via WhatsApp once a session concludes.
- **View Attendance & Analytics:** An interactive data grid that queries the MySQL tables to display comprehensive attendance ledgers for the entire institution.
- **Download Excel File Utility:** A functional component that allows the Administrator to dynamically export database records into structured spreadsheets for official university audits and offline archiving.

3.1.2 FACULTY DASHBOARD AND ITS COMPONENTS

Role of the Faculty: The Faculty module represents the primary operational engine of the system's daily lifecycle. Faculty members are the active session managers who interface with the biometric software directly within the classroom environment. Their role is to initiate the tracking sequence, monitor the automated data collection, and ensure the anti-bunking protocols (WhatsApp notifications) are successfully executed at the end of their respective lectures.

Components: The Faculty dashboard mirrors the Admin dashboard precisely, granting faculty members complete autonomy over their daily classroom management:

- **Mark Attendance Module:** This is the most frequently utilized component by the faculty. It initiates the OpenCV video capture, maps facial encodings, and updates the daily database ledger dynamically as students enter the classroom.
- **WhatsApp Automation Interface:** Following the completion of the biometric scan, the faculty utilizes this component to trigger the script that cross-references the roster and dispatches immediate absentee alerts to parents, actively enforcing the anti-bunking policy.
- **Register Student Module:** Faculty are granted access to this module to seamlessly onboard new students, update existing biometric profiles if a student's facial features change significantly, or correct typographical errors in the database without needing to submit a request to the system Admin.
- **View Attendance & Excel Export:** Faculty utilize this data grid to review the attendance metrics for their specific sessions, verify database updates, and download excel reports to submit to the academic department head at the end of the month.

3.1.3 STUDENT DASHBOARD AND ITS COMPONENTS

Role of the Student: The Student role is strictly defined as an end-user within the ecosystem. Their primary function is to monitor their own academic standing and track their attendance metrics. To maintain absolute data integrity and prevent unauthorized manipulation of the biometric ledgers, the student role is entirely read-only.

Components: By bypassing the administrative and scanning utilities, the Student Dashboard provides a streamlined, personalized portal focused solely on data visualization:

- **Personalized Analytics Display:** Upon login, the system queries the MySQL database using the student's unique session credentials. It dynamically calculates and displays key metrics, including the Total Working Days, the number of Days Present, and the number of Days Absent.
- **Real-time Percentage Tracker:** A critical metric component that calculates the student's overall attendance percentage. This allows students to instantly see if they are falling below the university's mandated attendance threshold, encouraging proactive academic discipline.
- **Lack of Write Privileges:** Unlike the Admin and Faculty modules, the Student dashboard intentionally lacks the "Register Student," "Mark Attendance," or "Download Excel" utilities, ensuring the system remains completely secure against unauthorized student interference.

CHAPTER IV

4 SYSTEM REQUIREMENTS

4.1 HARDWARE SPECIFICATIONS

Processor	: 13th Gen Intel(R) Core(TM) i5-1335U (1.30 GHz)
Installed RAM	: 16.0 GB (15.7 GB usable)
System type	: 64-bit operating system, x64-based processor
Camera	: Standard 720p or 1080p webcam
Storage	: 933 GB

4.2 SOFTWARE SPECIFICATIONS

Operating System	: Windows 11
Browser	: Google Chrome
Editor	: Visual Studio Code
Programming Language	: Python 3.9+
Frontend Framework	: Streamlit
Backend Database	: MySQL Server 8.0+
Database Interface	: MySQL Command Line Client
Virtual Environment	: Anaconda or standard Python venv

CHAPTER V

5 SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE DIAGRAM

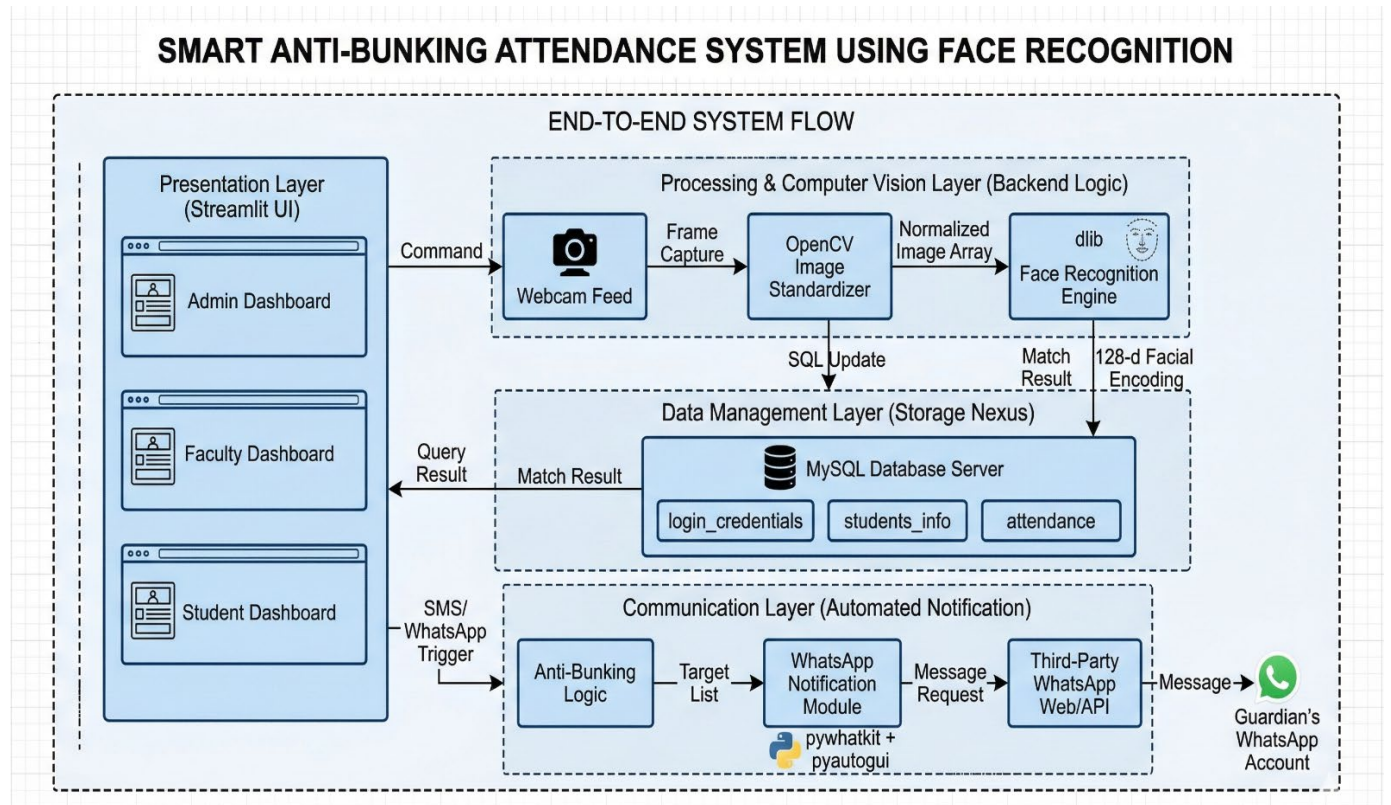
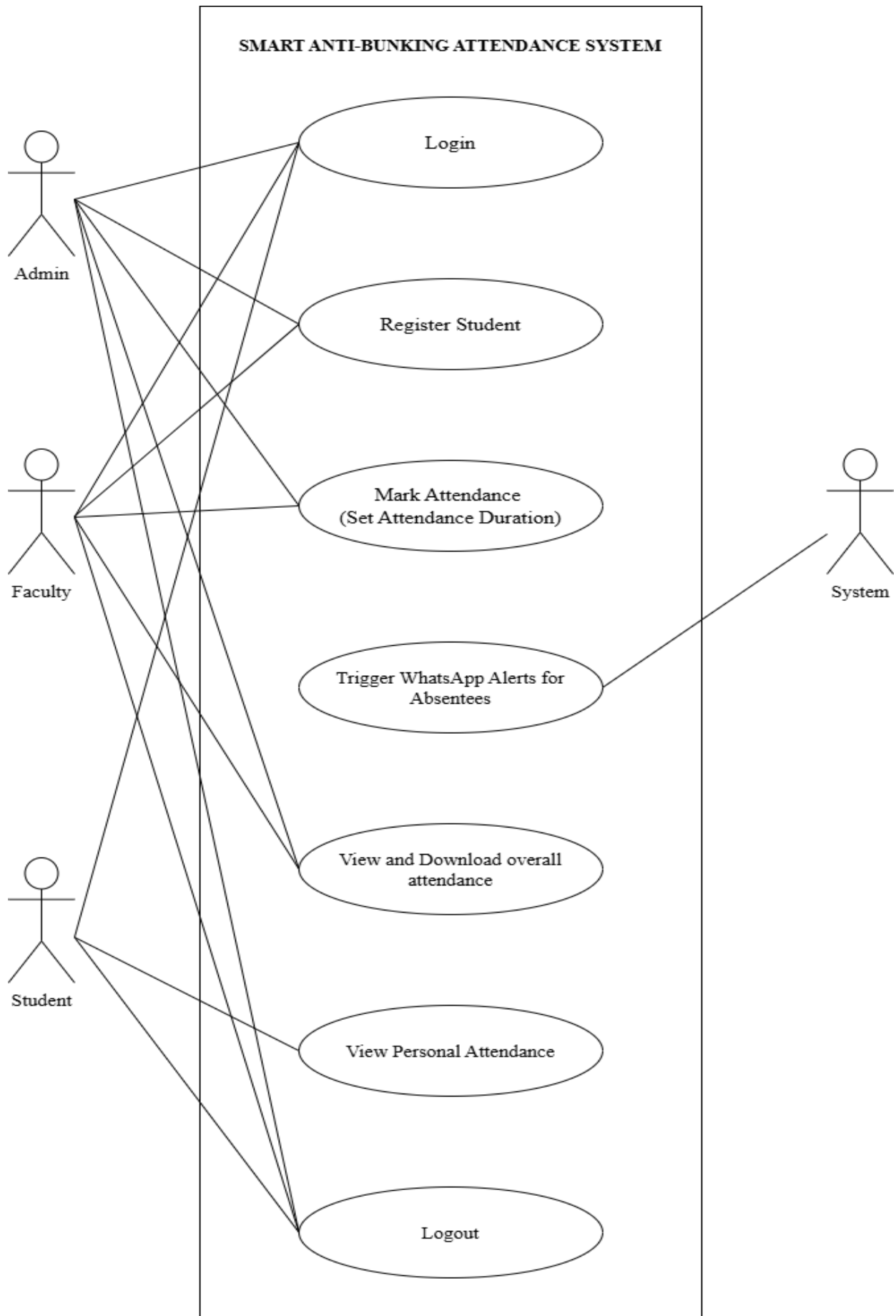


Figure 5.1: System Architecture of the Smart Anti-Bunking Attendance System

5.2 USE CASE DIAGRAM



CHAPTER VI

6 FUNCTIONAL REQUIREMENTS

6.1 TABLE STRUCTURE

6.1.1 LOGIN CREDENTIALS

TABLE DESCRIPTION: Contains login credentials of all administrators, faculties and students along with their name and designation to effectively provide Roll-Based Access to the Smart Anti-Bunking Attendance System. It also stores data of new students dynamically from the ‘Register Student’ form.

COLUMN NAME	DATA TYPE	CONSTRAINTS
name	VARCHAR(20)	NULL
designation	VARCHAR(10)	NULL
username	VARCHAR(15) PRIMARY KEY	NOT NULL
password	VARCHAR(255)	NOT NULL

6.1.2 STUDENTS_INFO

TABLE DESCRIPTION: Stores all the required information of the students such as Register Number, Name, Department, Parent’s WhatsApp Number and Face Encoding retrieved from the student’s uploaded image.

COLUMN NAME	DATA TYPE	CONSTRAINTS
RegNo	INTEGER PRIMARY KEY	NOT NULL
Name	VARCHAR(20)	NULL
Department	VARCHAR(20)	NULL
Parent_WhatsApp	VARCHAR(15)	NULL
Face_Encoding	BLOB	NULL

6.1.3 ATTENDANCE

TABLE DESCRIPTION: Stores the attendance of the students on a daily basis under a dynamic column name which is in the format “DD_MM_YY”. Attendance is stored as ‘0’ for “Absent” and ‘1’ for “Present”.

COLUMN NAME	DATA TYPE	CONSTRAINTS
RegNo	INTEGER PRIMARY KEY	NOT NULL
Name	VARCHAR(20)	NULL
DD_MM_YY	INTEGER	NULL

CHAPTER VII

7 SOFTWARE ENVIRONMENT

The software environment forms the technological backbone of the Smart Anti-Bunking Attendance System. Developing a real-time computer vision application that seamlessly integrates deep learning algorithms, database management, and automated external communication requires a highly optimized and interoperable technology stack. This chapter details the specific programming languages, frameworks, libraries, and database tools utilized to construct the system's architecture, justifying their selection for this Master's level implementation.

7.1 BACKEND PROGRAMMING LANGUAGE: PYTHON

Python serves as the core computational engine and backend logic handler for the entire application. It was selected over other compiled languages like Java or C++ due to its unparalleled ecosystem for artificial intelligence and data science.

Advantages and Purpose:

- **Extensive Machine Learning Support:** Python natively supports complex mathematical modeling and provides seamless bindings to low-level C++ libraries (like OpenCV and dlib), allowing the system to achieve high-performance biometric processing without sacrificing code readability.
- **Rapid Development and Integration:** Python's concise syntax enables rapid prototyping and the efficient integration of disparate system components, from algorithmic facial extraction to database connectivity.
- **Dynamic Memory Management:** The language dynamically handles the massive memory arrays required for real-time video frame processing, ensuring the system remains stable during continuous classroom scanning phases.

7.2 FRONTEND FRAMEWORK: STREAMLIT

Instead of employing a traditional, disjointed web stack (such as HTML/CSS/JavaScript coupled with a Django or Flask backend), this project utilizes Streamlit. Streamlit is an open-source Python framework specifically engineered for machine learning and data science applications.

Advantages and Purpose:

- **Native Python Integration:** Streamlit allows the entire Graphical User Interface (GUI) to be written purely in Python. This eliminates the need for complex API routing between the frontend and backend, drastically reducing latency.
- **Session State Management:** The framework natively handles user session states, which is critical for implementing the strict Role-Based Access Control (RBAC). It seamlessly routes users to the Admin, Faculty, or Student dashboards based on active login credentials.
- **Live Video Handling:** Streamlit provides highly optimized components to render real-time webcam feeds directly within the browser, dynamically stretching and updating frames without requiring manual page refreshes.

7.3 CORE LIBRARIES AND MODULES

The system's advanced functionalities are powered by a suite of specialized Python libraries. Each library acts as an isolated microservice, handling specific tasks within the operational lifecycle.

- **OpenCV (cv2):** The premier open-source computer vision library. In this project, OpenCV is utilized to interface directly with the hardware webcam, intercept the live video stream, and standardize the visual data. It captures frames in real-time and mathematically forces them into the strict, contiguous 8-bit RGB multi-channel arrays required by the deep learning models.
- **Face Recognition (face_recognition):** This library acts as a high-level wrapper for dlib, a state-of-the-art C++ machine learning toolkit. It is the biometric core of the project. It maps 128 distinct geometric landmarks on a human face, generating a highly accurate numerical encoding. This deep learning approach ensures accurate matching regardless of minor changes in lighting, facial hair, or viewing angles.
- **NumPy (numpy):** The fundamental package for scientific computing in Python. Because computer vision essentially treats images as massive grids of numbers, NumPy is deployed to handle the complex matrix mathematics and memory layout operations, ensuring the image byte data is perfectly formatted before being passed to the dlib engine.
- **PyWhatKit (pywhatkit):** The driver behind the anti-bunking mandate. Rather than relying on costly enterprise messaging APIs, pywhatkit is utilized to execute secure browser automation. It programmatically interfaces with WhatsApp Web to dynamically generate and dispatch instant absentee alerts to guardians, completely automating the institutional communication pipeline.
- **MySQL Connector (mysql-connector-python):** The secure bridge library that allows the Python backend to communicate with the relational database. It is responsible for executing SQL

queries dynamically, such as fetching user credentials or committing real-time "Present" statuses to the daily ledger.

- **OS and Datetime (os, datetime):** Standard Python utility modules used to handle localized file pathing (saving registered student images) and dynamically generating timestamped column names (e.g., DD_MM_YYYY) for the database attendance ledger.

7.4 DATABASE MANAGEMENT SYSTEM: MYSQL

Data integrity is paramount in any institutional software. The project utilizes MySQL Server as the backend Relational Database Management System (RDBMS), accessed and configured primarily via the MySQL Command Line Client.

Advantages and Purpose:

- **Strict Data Integrity:** MySQL enforces rigid schemas and constraints (such as Primary Keys on Registration Numbers), ensuring that duplicate records or proxy entries cannot compromise the academic ledger.
- **High-Speed Querying:** The database is highly optimized for read/write operations. When the Python backend continuously queries the database during the live video feed to match facial encodings, MySQL processes these transactions in milliseconds, preventing the camera feed from lagging.
- **Command Line Client Utilization:** The MySQL Command Line Client is utilized as the primary administrative tool for Data Definition Language (DDL) and Data Manipulation Language (DML) operations. By bypassing graphical overhead, the command-line interface provides developers and administrators with direct, lightweight, and absolute control over the login_credentials, students_info, and dynamic attendance tables, ensuring maximum security and execution speed.

CHAPTER VIII

8 SOFTWARE IMPLEMENTATION

app.py

```
import streamlit as st

import pandas as pd

from datetime import datetime, timedelta

import face_recognition

import cv2

import os


from database import authenticate_user, register_student, ensure_date_column_exists,
get_absentees_for_date, get_db_connection

from face_cv import run_attendance_camera

from whatsapp_alert import send_bulk_whatsapp_notifications


# --- CACHING FUNCTIONS ---

@st.cache_data(ttl=3600) # Cache for 1 hour

def cached_get_attendance_data():

    conn = get_db_connection()

    df = pd.read_sql("SELECT * FROM attendance", conn)

    conn.close()

    return df


@st.cache_data(ttl=1800) # Cache for 30 minutes
```

```

def cached_load_known_faces():

    from face_cv import load_known_faces

    return load_known_faces()


# Initialize Session State

if 'logged_in' not in st.session_state:

    st.session_state.logged_in = False

if 'user_role' not in st.session_state:

    st.session_state.user_role = None

if 'username' not in st.session_state:

    st.session_state.username = None


# --- AUTHENTICATION ---

def login_page():

    st.image("Assets\\Nevermore Academy Smart Attendance System.png")

    with st.form("login_form"):

        username = st.text_input("Username")

        password = st.text_input("Password", type="password")

        submitted = st.form_submit_button("Login")

    if submitted:

        user = authenticate_user(username, password)

        if user:

            st.session_state.logged_in = True

            st.session_state.user_role = user['designation']

```

```

        st.session_state.username = user['username']

        st.session_state.name = user['name']

        st.rerun()

    else:

        st.error("Invalid Username or Password")

# --- ADMIN / FACULTY DASHBOARD ---

def admin_dashboard():

    st.sidebar.title(f"Welcome {st.session_state.name}!")

    menu = st.sidebar.selectbox("Menu", ["Mark Attendance", "Register Student", "View Attendance"])

    if st.sidebar.button("Logout"):

        st.session_state.clear()

        st.rerun()

    if menu == "Mark Attendance":

        st.image("Assets\\Nevermore Academy Smart Attendance System.png")

        st.header("Start Attendance Session")

        col1, col2 = st.columns(2)

        with col1:

            duration = st.number_input("Session Duration (minutes)", min_value=1, max_value=60,
value=5)

        if st.button("Start Camera & Attendance"):

            date_str = datetime.now().strftime("%d-%m-%y")

```

```

ensure_date_column_exists(date_str)

end_time = datetime.now() + timedelta(minutes=duration)

st.info(f'Attendance running until {end_time.strftime("%H:%M:%S")}')

# Placeholder for camera feed

camera_placeholder = st.empty()

# Blocking call: Runs camera until end_time

marked_count = run_attendance_camera(date_str, end_time, camera_placeholder)

st.success(f'Attendance session ended! {marked_count} students marked present.')

# Trigger WhatsApp Notifications

with st.spinner("Identifying absentees and sending WhatsApp notifications..."):

    absentees = get_absentees_for_date(date_str)

    success_msgs = send_bulk_whatsapp_notifications(absentees)

    st.success(f'Sent {success_msgs} WhatsApp notifications to parents of absent students.')

elif menu == "Register Student":

    st.image("Assets\\Nevermore Academy Smart Attendance System.png")

    st.header("Register New Student")

    with st.form("register_form", clear_on_submit=True):

        regno_input = st.text_input("Register Number")

        name = st.text_input("Name")

```

```

dept = st.text_input("Department")

parent_whatsapp = st.text_input("Parent WhatsApp (+91...)")

username = st.text_input("Student Login Username")

password = st.text_input("Student Login Password", type="password")


image_file = st.file_uploader("Upload Student Photo", type=["jpg", "jpeg", "png"])

submit = st.form_submit_button("Register")


if submit and image_file:

    try:

        regno = int(regno_input)

        if not os.path.exists("Face_Encodings"):

            os.makedirs("Face_Encodings")

            # 1. HARD-SAVE the raw upload straight to your hard drive.

            temp_path = f"Face_Encodings/temp_{regno}.jpg"

            with open(temp_path, "wb") as f:

                f.write(image_file.getbuffer())

            # 2. Read it back directly from the hard drive using OpenCV.

            img = cv2.imread(temp_path)

            if img is None:

                st.error("The image file itself is corrupted. Please download a fresh JPG from Google
and try that.")

```

else:

3. OpenCV reads as BGR. Convert to standard RGB for dlib.

```
rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

4. Extract the face encoding

```
encodings = face_recognition.face_encodings(rgb_img)
```

if len(encodings) > 0:

```
    face_encoding = encodings[0]
```

5. Rename to the final path and clean up

```
final_path = f"Face_Encodings/{regno}.jpg"
```

```
cv2.imwrite(final_path, img)
```

```
if os.path.exists(temp_path):
```

```
    os.remove(temp_path)
```

6. Save to Database

```
success = register_student(
```

```
    regno, name, dept, parent_whatsapp, face_encoding, username, password
```

```
)
```

if success:

```
    st.success("Student Registered Successfully!")
```

else:

```
    st.error("Failed to save to database. Check if RegNo or Username already exists.")
```

```

        else:

            st.error("No face found in the uploaded image. Please try a clearer photo.")

    except Exception as e:

        st.error(f"Image processing error: {e}")

elif menu == "View Attendance":

    st.image("Assets\\Nevermore Academy Smart Attendance System.png")

    st.header("Attendance Reports")

    df = cached_get_attendance_data()

    date_str = datetime.now().strftime("%d-%m-%y")

    if date_str in df.columns:

        total = len(df)

        present = df[date_str].sum()

        absent = total - present

    st.subheader(f"Today's Statistics ( {date_str} )")

    c1, c2, c3 = st.columns(3)

    c1.metric("On Roll", total)

    c2.metric("Present", present)

    c3.metric("Absent", absent)

st.dataframe(df)

```

```

# Export to Excel

output_file = "attendance_report.xlsx"

df.to_excel(output_file, index=False)

with open(output_file, "rb") as file:

    st.download_button("Download as Excel", data=file, file_name=output_file,
mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")

# --- STUDENT DASHBOARD ---

def student_dashboard():

    st.sidebar.title(f'Welcome {st.session_state.name}!')

    if st.sidebar.button("Logout"):

        st.session_state.clear()

        st.rerun()

st.image("Assets\\Nevermore Academy Smart Attendance System.png")

st.header("My Attendance Dashboard")

conn = get_db_connection()

cursor = conn.cursor(dictionary=True)

cursor.execute("""

    SELECT a.* FROM attendance a

    JOIN login_credentials l ON a.name = l.name

    WHERE l.username = %s

""", (st.session_state.username,))

student_record = cursor.fetchone()

```

```
conn.close()
```

```
if student_record:
```

```
    # Extract dynamic date columns (keeping your original exact logic)
```

```
    #date_cols = [col for col in student_record.keys() if col not in ('regno', 'name')]
```

```
    # Securely fetch dictionary keys regardless of MySQL casing
```

```
    regno_key = 'RegNo' if 'RegNo' in student_record else 'regno'
```

```
    name_key = 'Name' if 'Name' in student_record else 'name'
```

```
    regno = student_record[regno_key]
```

```
    name = student_record[name_key]
```

```
    # --- THE FIX FOR ISSUE 2 ---
```

```
    # Strictly filter: Only count columns that start with a digit (like 09-03-26)
```

```
    # This prevents it from accidentally counting 'id', 'dept', etc.
```

```
    date_cols = [
```

```
        col for col in student_record.keys()
```

```
        if col.lower() not in ('regno', 'name') and str(col)[0].isdigit()
```

```
    ]
```

```
    total_days = len(date_cols)
```

```
    days_present = sum([student_record[col] for col in date_cols if student_record[col] == 1])
```

```
    days_absent = total_days - days_present
```

```
    attendance_percentage = (days_present / total_days * 100) if total_days > 0 else 0
```

```

# --- NEW UI: Image on the left, Stats on the right ---

col_img, col_info = st.columns([1, 3])

with col_img:

    # Construct the image path using the exact RegNo from the database

    student_regno = student_record['RegNo']

    image_path = f"Face_Encodings/{student_regno}.jpg"

    if os.path.exists(image_path):

        st.image(image_path, width=150, caption="Profile Photo")

    else:

        st.warning("Photo not found.")

with col_info:

    # Displaying your original headers and metrics

    st.subheader(f"RegNo: {student_record['RegNo']} | Name: {student_record['Name']}")

    c1, c2, c3, c4 = st.columns([1.5, 1, 1, 1.2])

    c1.metric("Total Working Days", total_days)

    c2.metric("Days Present", days_present)

    c3.metric("Days Absent", days_absent)

    c4.metric("Attendance %", f"{attendance_percentage:.1f}%")

else:

```

```

        st.warning("No attendance records found.")

# --- ROUTING LOGIC ---

if not st.session_state.logged_in:

    login_page()

else:

    if st.session_state.user_role in ['Admin', 'Faculty']:

        admin_dashboard()

    elif st.session_state.user_role == 'Student':

        student_dashboard()

```

database.py

```

import mysql.connector

import pandas as pd

from datetime import datetime


DB_CONFIG = {

    'host': 'localhost',

    'user': 'root',

    'password': 'cs@123',

    'database': 'smart_attendance'

}


def get_db_connection():

    return mysql.connector.connect(**DB_CONFIG)


def authenticate_user(username, password):

```

```

conn = get_db_connection()

cursor = conn.cursor(dictionary=True)

cursor.execute("SELECT * FROM login_credentials WHERE username=%s AND password=%s",
(username, password))

user = cursor.fetchone()

conn.close()

return user

```

```

def register_student(regno, name, dept, whatsapp, encoding, username, password):

```

```

    conn = get_db_connection()

    cursor = conn.cursor()

    try:

        # Convert numpy encoding to JSON for storage

        encoding_blob = encoding.tobytes()

        # 1. Insert into students_info

        cursor.execute("""

            INSERT INTO students_info (RegNo, Name, Department, Parent_WhatsApp, Face_Encoding)

            VALUES (%s, %s, %s, %s, %s)

            """, (regno, name, dept, whatsapp, encoding_blob))

        # 2. Insert into login_credentials

        cursor.execute("""

            INSERT INTO login_credentials (name, username, password, designation)

            VALUES (%s, %s, %s, 'Student')

            """, (name, username, password))

```

```

# 3. Initialize in attendance table

cursor.execute("INSERT INTO attendance (RegNo, Name) VALUES (%s, %s)", (regno, name))

conn.commit()

return True

except Exception as e:

    print(f"Error: {e}")

    conn.rollback()

    return False

finally:

    conn.close()

def ensure_date_column_exists(date_str):

    """Creates a new column for today's date if it doesn't exist."""

    conn = get_db_connection()

    cursor = conn.cursor()

    try:

        cursor.execute(f"ALTER TABLE attendance ADD COLUMN `{date_str}` INT DEFAULT 0")

        conn.commit()

    except mysql.connector.Error as err:

        # Error 1060 indicates duplicate column name (column already exists)

        if err.errno != 1060:

            raise err

    finally:

```

```
conn.close()
```

```
def mark_student_present(regno, date_str):
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute(f"UPDATE attendance SET `{date_str}` = 1 WHERE RegNo = %s", (regno,))
```

```
    conn.commit()
```

```
    conn.close()
```

```
def get_absentees_for_date(date_str):
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    query = f"""
```

```
        SELECT a.RegNo, a.Name, s.Parent_WhatsApp
```

```
        FROM attendance a
```

```
        JOIN students_info s ON a.RegNo = s.RegNo
```

```
        WHERE a.`{date_str}` = 0
```

```
    """
```

```
    cursor.execute(query)
```

```
    absentees = cursor.fetchall()
```

```
    conn.close()
```

```
    return absentees
```

face_cv.py

```
import cv2
```

```
import face_recognition
```

```

import numpy as np

import json

import streamlit as st

from database import get_db_connection, mark_student_present

from datetime import datetime


@st.cache_data(ttl=1800) # Cache for 30 minutes

def load_known_faces():

    conn = get_db_connection()

    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT RegNo, Name, Face_Encoding FROM students_info")

    records = cursor.fetchall()

    conn.close()


    known_encodings = []

    known_regno = []

    known_names = []


    for row in records:

        if row['Face_Encoding']:

            encoding = np.frombuffer(row['Face_Encoding'], dtype=np.float64)

            known_encodings.append(encoding)

            known_regno.append(row['RegNo'])

            known_names.append(row['Name'])

```

```

    return known_encodings, known_regnos, known_names

def run_attendance_camera(date_str, end_time, placeholder):
    """
    Runs the webcam, compares faces, updates DB, and stops at end_time.
    Uses a Streamlit placeholder to render frames without rerunning the whole script.
    """
    known_encodings, known_regnos, known_names = load_known_faces()

    cap = cv2.VideoCapture(0)

    # Keep track of who has been marked to avoid spamming the DB
    marked_today = set()

    while datetime.now() < end_time:
        ret, frame = cap.read()

        if not ret:
            st.error("Failed to access webcam.")
            break

        # Resize frame for faster processing
        small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
        rgb_small_frame = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)

        # Find faces
        face_locations = face_recognition.face_locations(rgb_small_frame)

```

```

face_encodings = face_recognition.face_encodings(rgb_small_frame, face_locations)

for face_encoding, (top, right, bottom, left) in zip(face_encodings, face_locations):

    matches = face_recognition.compare_faces(known_encodings, face_encoding)

    name = "Unknown"

    if True in matches:

        first_match_index = matches.index(True)

        regno = known_regnos[first_match_index]

        name = known_names[first_match_index]

        # Mark attendance if not already marked in this session

        if regno not in marked_today:

            mark_student_present(regno, date_str)

            marked_today.add(regno)

        # Scale back up face locations and draw box

        top *= 4; right *= 4; bottom *= 4; left *= 4

        cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2)

        cv2.putText(frame, name, (left + 6, bottom - 6), cv2.FONT_HERSHEY_DUPLEX, 0.5, (255,
255, 255), 1)

    # Display the frame in Streamlit

    placeholder.image(frame, channels="BGR", width="stretch")

cap.release()

```

```
placeholder.empty() # Clear camera feed when done
```

```
return len(marked_today)
```

whatsapp_alert.py

```
import pywhatkit as kit
```

```
import time
```

```
import pyautogui
```

```
from pynput.keyboard import Key, Controller
```

```
keyboard = Controller()
```

```
def send_bulk_whatsapp_notifications(absentees):
```

```
    """
```

```
    Sends messages using pywhatkit by automating the browser.
```

```
    Note: Requires WhatsApp Web to be logged in on the default browser.
```

```
    """
```

```
    success_count = 0
```

```
    for student in absentees:
```

```
        name = student['Name']
```

```
        regno = student['RegNo']
```

```
        # Ensure number starts with '+' and country code, e.g., '+911234567890'
```

```
        phone_number = student['Parent_WhatsApp']
```

```
        message = (
```

```
            f'Dear Parent, "
```

```

f"Your ward {name} (Reg No: {regno}) was absent today."

f"-- Automated Attendance System"

)

try:

    # sendwhatmsg_instantly parameters:

    # phone_no, message, wait_time (seconds), tab_close (bool), close_time (seconds)

    kit.sendwhatmsg_instantly(

        phone_no=phone_number,

        message=message,

        wait_time=15, # Time for WhatsApp Web to load

        tab_close=True,

        close_time=3

    )

    # pywhatkit sometimes needs a small manual 'Enter' trigger depending on the OS/Browser

    time.sleep(2)

    pyautogui.press('enter') # Simulate pressing 'Enter' to send the message

    #keyboard.press(Key.enter)

    #keyboard.release(Key.enter)

    success_count += 1

    # Add a buffer between students to avoid WhatsApp spam detection

    time.sleep(5)

```

```
.  
    except Exception as e:  
        print(f'Error sending to {name}: {e}')  
  
return success_count
```

CHAPTER IX

SAMPLE SCREENS

Fig 9.1: LOGIN SCREEN

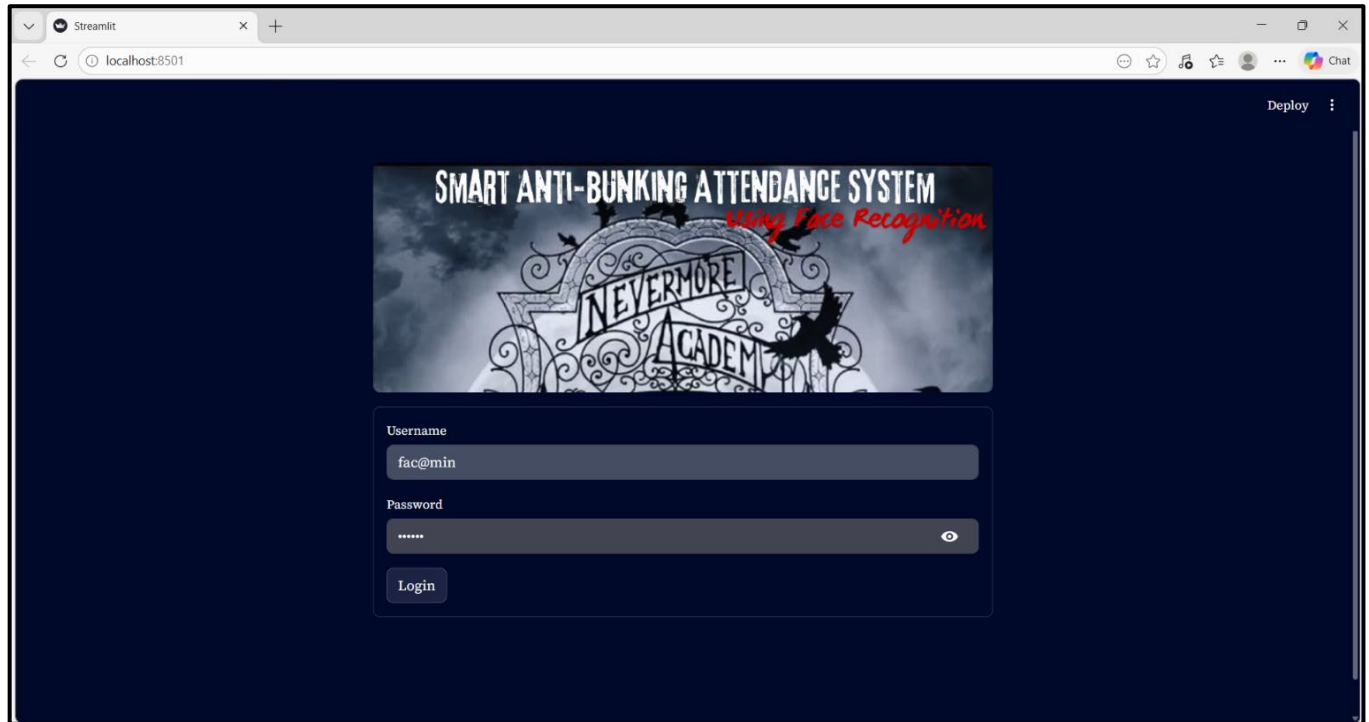


Fig 9.2: ADMIN/FACULTY DASHBOARD WITH MARK ATTENDANCE MODULE

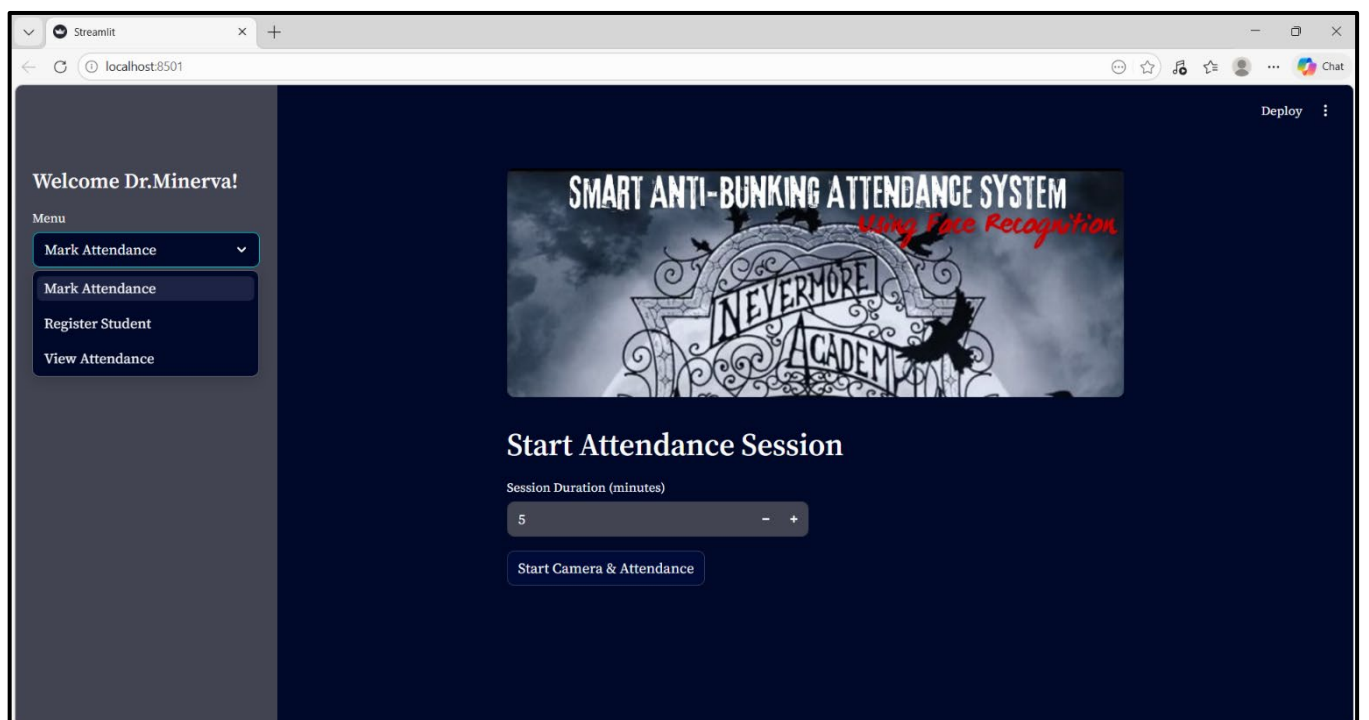


Fig 9.3: LIVE FACE RECOGNITION BASED ATTENDANCE

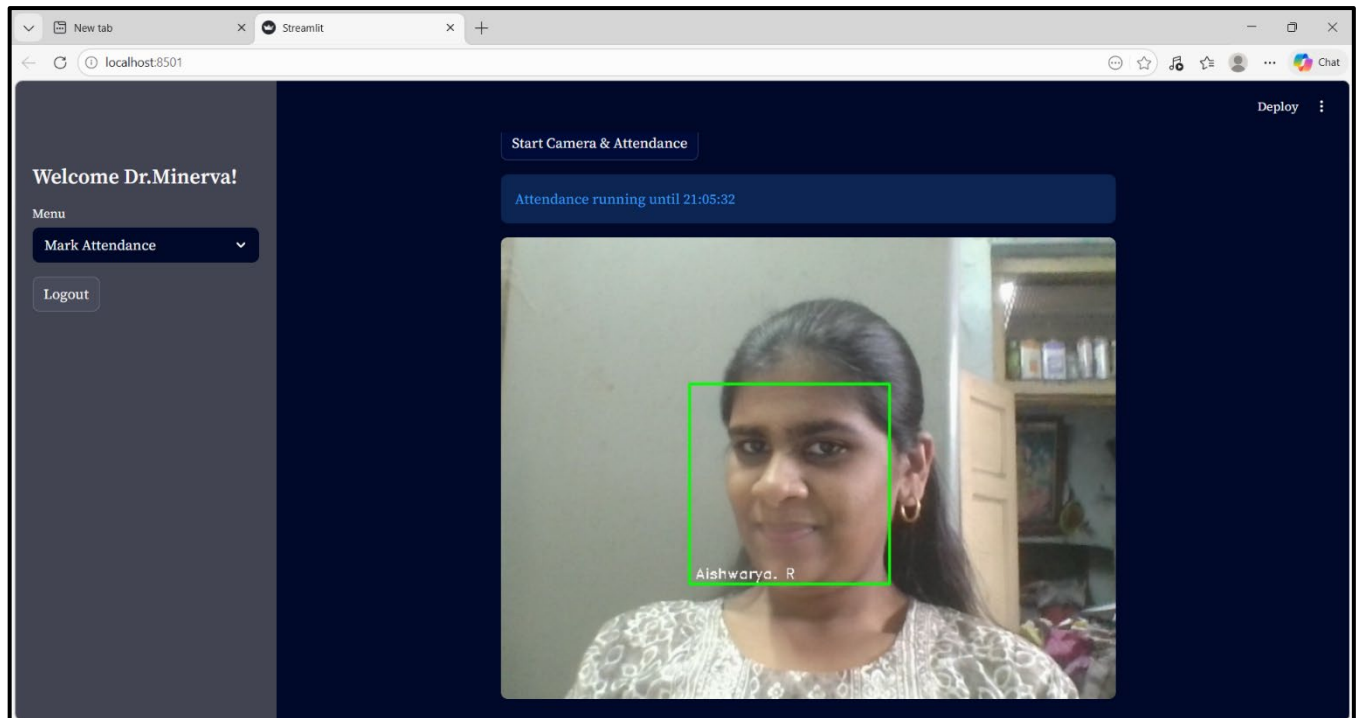


Fig 9.4: CLOSURE OF ATTENDANCE

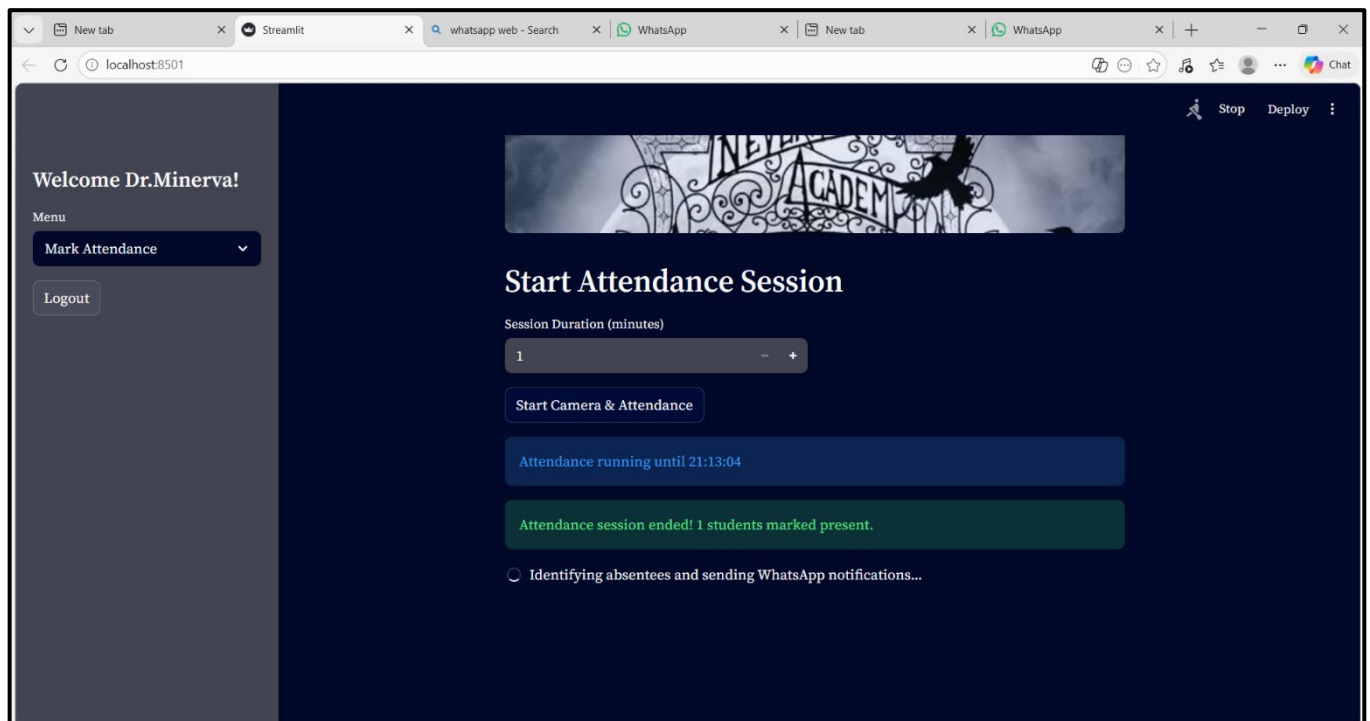


Fig 9.5: AUTOMATED WHATSAPP ALERT

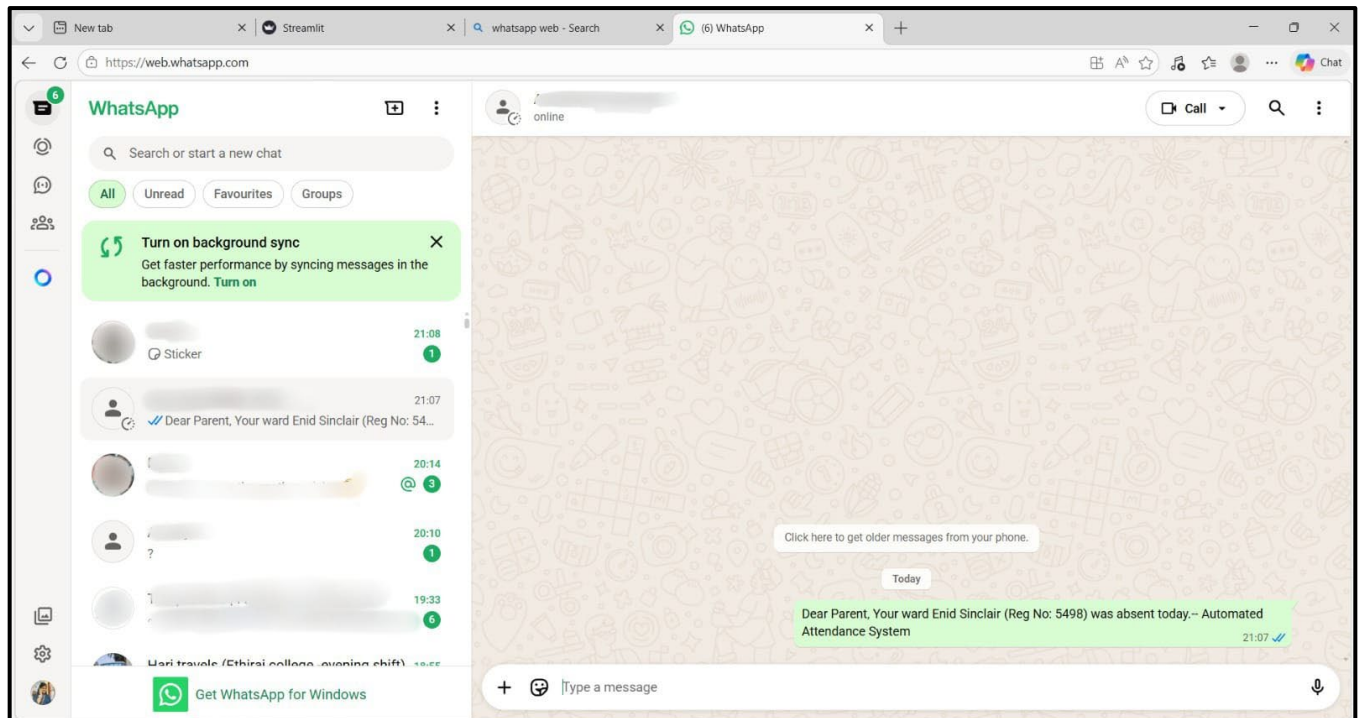


Fig 9.6: REGISTER STUDENT MODULE

Welcome Dr.Minerva!

Menu

Register Student

Logout

SMART ANTI-BUNKING ATTENDANCE SYSTEM

Using Face Recognition

NEVERMORE ACADEMY

Register New Student

Register Number
1372

Name
Wednesday Addams

Department
Forensic Sciences

Parent WhatsApp (+91...)
+919444045984

Student Login Username
stu@wed

Student Login Password
.....

Upload Student Photo
Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG
Browse files

Wednesday Addams.jpg 59.6KB

Register

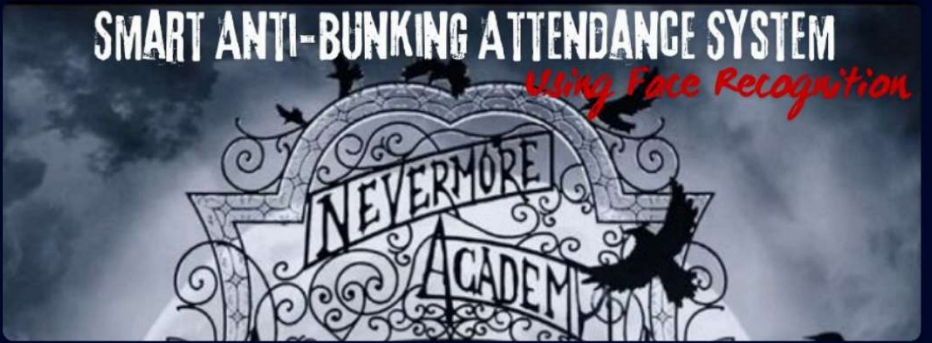
Fig 9.7: VIEW ATTENDANCE MODULE

Welcome Dr.Minerva!

Menu

View Attendance ▾

Logout



Attendance Reports

Today's Statistics (11-03-26)

On Roll	Present	Absent
9	5	4

	RegNo	Name	09-03-26	10-03-26	11-03-26
0	1256	Yoko Tanaka	1	1	1
1	2003	Aarthy C	0	1	1
2	3123	Aishwarya. R	1	1	1
3	3168	Draco Malfoy	0	0	0
4	5025	Ginny Weasley	0	0	0
5	5498	Enid Sinclair	1	0	1
6	7078	Harry Potter	1	1	0
7	7152	Xavier Thorpe	0	0	0
8	8211	Hermione Granger	0	0	1

Download as Excel

Fig 9.8: DOWNLOADED EXCEL REPORT

	A	B	C	D	E	F
1	RegNo	Name	09-03-26	10-03-26	11-03-26	
2	1256	Yoko Tanaka	1	1	1	
3	2003	Aarthy C	0	1	1	
4	3123	Aishwarya. R	1	1	1	
5	3168	Draco Malfoy	0	0	0	
6	5025	Ginny Weasley	0	0	0	
7	5498	Enid Sinclair	1	0	1	
8	7078	Harry Potter	1	1	0	
9	7152	Xavier Thorpe	0	0	0	
10	8211	Hermione Granger	0	0	1	
11						

Fig 9.9: STUDENT DASHBOARD – STUDENT’S PERSONAL ATTENDANCE

The screenshot shows a web browser window with the address bar displaying 'localhost:8501'. The page has a dark blue background. On the left, a sidebar contains the text 'Welcome Aishwarya. R!' and a 'Logout' button. The main content area features a banner for 'SMART ANTI-BUNKING ATTENDANCE SYSTEM Using Face Recognition' with a gothic-style logo for 'NEVERMORE ACADEMY'. Below the banner, the title 'My Attendance Dashboard' is followed by the user's details: 'RegNo: 3123 | Name: Aishwarya. R'. A table displays attendance statistics, and a profile photo of the student is shown below the table.

Welcome Aishwarya. R!

Logout

SMART ANTI-BUNKING ATTENDANCE SYSTEM
Using Face Recognition

NEVERMORE ACADEMY

My Attendance Dashboard

RegNo: 3123 | Name: Aishwarya. R

Total Working Days	Days Present	Days Absent	Attendance %
3	3	0	100.0%

Profile Photo

CHAPTER X

10 USER MANUAL

10.1 ACCESS AND AUTHENTICATION (LOGIN PORTAL)

The system uses a secure, role-based login screen to ensure users only access the tools meant for them.

1. Open the application in your web browser.
2. Select your designated role from the dropdown menu: Admin, Faculty, or Student.
3. Enter your assigned Username and Password.
4. Click the Login button. The system will securely authenticate your credentials and load your specific dashboard.

10.2 ADMIN & FACULTY OPERATIONS

Administrators and Faculty members share the same tools to manage classes and enforce attendance policies.

10.2.1 Registering a New Student

Use this section to add new students to the biometric database.

1. Navigate to the "Register Student" tab on your dashboard.
2. Fill in the student's details: *Register Number, Name, Department, and Parent's WhatsApp Number* (include the country code).
3. Upload Photograph: Click the "Browse files" button. Select a clear, well-lit, front-facing photograph of the student (.jpg or .png). *Note: The system extracts the facial data directly from this uploaded file.*
4. Click "Register". A success message will confirm the student is added to the system.

10.2.2 Marking Attendance

Use this module during class to track student presence automatically.

1. Navigate to the "Mark Attendance" tab.
2. Input a duration for the camera to mark the attendance.

3. The live webcam feed will appear. As students enter the camera's view, the system will instantly recognize their faces and log them as "Present" in the background.
4. Upon the closure of the live attendance, immediately alert notifications will be triggered to the WhatsApp number of the parents of the absentees. Do not touch your mouse or keyboard until the system finishes sending all alerts, as it relies on automated keystrokes to dispatch the texts.

10.2.3 Viewing and Exporting Reports

1. Navigate to the "View Attendance" tab to see the complete attendance ledger in a grid.
2. To save a copy for offline records, click the "Download Excel" button located beneath the table.

10.3 STUDENT OPERATIONS

The Student dashboard is a secure, personalized portal for tracking academic standing.

1. Log in using your Student credentials.
2. Your personalized dashboard will load, displaying your uploaded profile photograph.
3. Review your real-time attendance metrics, which include:
 - Total Working Days
 - Days Present
 - Days Absent
 - Overall Attendance Percentage
4. Keep an eye on your percentage to ensure it remains above the university's required threshold!

CHAPTER XI

CONCLUSION

The Smart Anti-Bunking Attendance System successfully modernizes and secures the academic administrative framework by bridging the gap between advanced biometric technology and automated communication. By replacing fallible, manual roll calls and physical ledgers with a highly accurate, real-time computer vision pipeline, this project fundamentally eliminates the pervasive issues of proxy attendance, data manipulation, and the loss of valuable instructional time.

The integration of deep learning algorithms (dlib) and hardware-level video processing (OpenCV) ensures that a student's physical presence is computationally verified against a secure MySQL database before any academic credit is awarded. Furthermore, the system transitions attendance tracking from a passive recording chore into an active, disciplinary tool. By leveraging browser automation (pywhatkit) to dispatch instantaneous absentee alerts via WhatsApp, the system completely dismantles the culture of "class bunking." Guardians are no longer kept in the dark until the end of a semester; instead, they are immediately integrated into the daily educational loop.

Ultimately, this implementation provides educational institutions with a robust, scalable, and tamper-proof ecosystem. It enforces strict student accountability, optimizes faculty workload, and fosters a highly transparent, disciplined academic environment.

While the current architecture successfully fulfils its primary anti-bunking and biometric tracking mandates, software systems must continuously evolve to address emerging edge cases. The following enhancements are proposed for future iterations of the project to scale its utility and harden its security:

- **Automated Academic Result Notifications:** The existing automated WhatsApp communication pipeline is highly effective for attendance, but its utility can be significantly expanded. Future iterations will integrate the university's internal grading database with the messaging module. This will allow the system to automatically generate and dispatch semester grades, internal assessment marks, and critical academic warnings directly to parents, creating a unified, centralized communication hub.
- **Dynamic Sender Rotation (Anti-Blocking Mechanism):** A highly anticipated real-world challenge is user interference. To prevent guardians from seeing absentee alerts, students may attempt to gain access to their parents' devices to "block" or "report" the automated sender's WhatsApp number. To circumvent this, the system will implement a dynamic sender rotation

protocol. By utilizing a rotating pool of registered outgoing contact numbers, the system will periodically shift the sender ID. This ensures that even if one number is blocked by a student, subsequent notifications will successfully bypass the blocklist via a newly assigned number.

- **Liveness and Anti-Spoofing Detection:** To further harden the biometric security against proxy attempts, the computer vision pipeline can be upgraded to include liveness detection algorithms. By analyzing depth data, eye-blinking patterns, or micro-expressions via the webcam, the system will be able to differentiate between a live human face and a high-resolution photograph or video held up to the camera, neutralizing sophisticated spoofing attempts.

BIBLIOGRAPHY

- [1] Joba, O. (n.d.). *Building an Automated Attendance System Using Face Recognition and MySQL*. Medium. Retrieved from: <https://medium.com/@oluwabori.jobba/building-an-automated-attendance-system-using-face-recognition-and-mysql-4e7a0d02502a>
- [2] Scaler Topics. (n.d.). *Smart Attendance System Using Face Recognition*. Retrieved from: <https://www.scaler.com/topics/smart-attendance-system-using-face-recognition/>
- [3] ResearchGate. (2019). *Smart Attendance Management System Using Face Recognition*. Publication No. 334820129. Retrieved from: https://www.researchgate.net/publication/334820129_Smart_Attendance_Management_System_Using_Face_Recognition
- [4] Tech Learners Journal. (n.d.). *Building an Automated Employee Attendance System with Face Recognition*. Medium. Retrieved from: <https://medium.com/tech-learners-journal/building-an-automated-employee-attendance-system-with-face-recognition-d923c5e3b682>
- [5] GeeksforGeeks. (n.d.). *Face Recognition Using Artificial Intelligence*. Machine Learning Portal. Retrieved from: <https://www.geeksforgeeks.org/machine-learning/face-recognition-using-artificial-intelligence/>
- [6] GeeksforGeeks. (n.d.). *Automate WhatsApp Messages with Python Using pywhatkit Module*. Python Portal. Retrieved from: <https://www.geeksforgeeks.org/python/automate-whatsapp-messages-with-python-using-pywhatkit-module/>
- [7] A. Jadhav, A. Jadhav, T. Ladhe, and K. Yeolekar. (2017). *Automated Attendance System Using Face Recognition*. International Research Journal of Engineering and Technology (IRJET), Volume 4, Issue 1. Retrieved from: <https://www.irjet.net>
- [8] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- [9] King, D. E. (2009). *Dlib-ml: A Machine Learning Toolkit*. Journal of Machine Learning Research, 10, 1755-1758.
- [10] Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Retrieved from: <https://dev.mysql.com/doc/refman/8.0/en/>